

```

#include <stdio.h>
#include <stdlib.h>

void merge(int arr[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = 0;
    int size = high - low + 1;
    int *local_temp = (int *)malloc(size * sizeof(int));
    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            local_temp[k++] = arr[i++];
        } else {
            local_temp[k++] = arr[j++];
        }
    }
    while (i <= mid) {
        local_temp[k++] = arr[i++];
    }
    while (j <= high) {
        local_temp[k++] = arr[j++];
    }
    for (i = 0; i < size; i++) {
        arr[low + i] = local_temp[i];
    }
    free(local_temp);
}

```

```

void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = low + (high - low) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

```

```

int main() {
    int n;
    scanf("%d", &n);

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}

```

```

}

mergeSort(arr, 0, n - 1);

for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}

free(arr);
return 0;
}

```

tejas.mahurkar.batch2025@sitnagpur.siu.edu.in Support Logout

merge.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void merge(int arr[], int low, int mid, int high) {
5     int i = low;
6     int j = mid + 1;
7     int k = 0;
8     int size = high - low + 1;

```

Average time: 0.068 s (68.40 ms) Maximum time: 0.075 s (75.00 ms)

3 out of 3 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (75 ms) Debug

Expected output	Actual output
5	5
9 -5 7 -2 -1	9 -5 7 -2 -1
-5 -2 -1 7 9	-5 -2 -1 7 9

Test case 2 (67 ms) Debug

Expected output	Actual output
7	7
2 9 1 6 3 10 7	2 9 1 6 3 10 7
1 2 3 6 7 9 10	1 2 3 6 7 9 10

Test case 3 (65 ms) Debug

Expected output	Actual output
5	5
3 7 4 1 5	3 7 4 1 5
1 3 4 5 7	1 3 4 5 7

Terminal Test cases